Automated Program Repair: Emerging Trends Pose and Expose Problems for Benchmarks

JOSEPH RENZULLO, School of Computing and Augmented Intelligence, Arizona State University, Tempe, United States
PEMMA REITER, Arizona State University, Tempe, United States
WESTLEY WEIMER, University of Michigan, Ann Arbor, United States
STEPHANIE FORREST, Arizona State University, Tempe, United States

Machine learning (ML) pervades the field of Automated Program Repair (APR). Algorithms deploy neural machine translation and large language models (LLMs) to generate software patches, among other tasks. But, there are important differences between these applications of ML and earlier work, which complicates the task of ensuring that results are valid and likely to generalize. A challenge is that the most popular APR evaluation benchmarks were not designed with ML techniques in mind. This is especially true for LLMs, whose large and often poorly-disclosed training datasets may include problems on which they are evaluated.

This article reviews work in APR published in the field's top five venues since 2018, emphasizing emerging trends in the field, including the dramatic rise of ML models, including LLMs. ML-based articles are categorized along structural and functional dimensions, and a variety of issues are identified that these new methods raise. Importantly, data leakage and contamination concerns arise from the challenge of validating ML-based APR using existing benchmarks, which were designed before these techniques were popular. We discuss inconsistencies in evaluation design and performance reporting and offer pointers to solutions where they are available. Finally, we highlight promising new directions that the field is already taking.

CCS Concepts: • **Software and its engineering → Search-based software engineering**; **Maintaining software**; • **Computing methodologies → Machine learning algorithms;**

Additional Key Words and Phrases: automated program repair, machine learning, benchmarks, patch quality

ACM Reference Format:
Joseph Renzullo, Pemma Reiter, Westley Weimer, and Stephanie Forrest. 2025. Automated Program Repair: Emerging Trends Pose and Expose Problems for Benchmarks. *ACM Comput. Surv.* 57, 8, Article 208 (March 2025), 18 pages. https://doi.org/10.1145/3704997

Joseph Renzullo and Pemma Reiter contributed equally to this research.
Stephanie Forrest Also at the Santa Fe Institute.

Authors' Contact Information: Joseph Renzullo, School of Computing and Augmented Intelligence, Arizona State University, Tempe, Arizona, United States; e-mail: renzullo@asu.edu; Pemma Reiter, Arizona State University, Tempe, Arizona, United States; e-mail: pdreiter@asu.edu; Westley Weimer, University of Michigan, Ann Arbor, Michigan, United States; e-mail: weimerw@umich.edu; Stephanie Forrest, Arizona State University, Tempe, Arizona, United States; e-mail: steph@asu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0360-0300/2025/03-ART208
https://doi.org/10.1145/3704997

1 Introduction

Automated Program Repair (**APR**), a subfield of software engineering, aims at reducing or eliminating direct human involvement in the repair of software defects, or bugs. A variety of techniques have been developed, e.g., evolutionary computation [61, 135], methods incorporating templated mutation operators [72], semantic inference techniques [80] targeting single-cause defects, and methods designed to handle multi-edit bugs [103]. Recently, ML-based methods have also been adopted (Section 3), raising several issues, including data leakage (Section 4), lack of a commonly accepted formal definition of APR, and widely used benchmarks that are insufficient for standardized evaluations of ML-based methods (Section 5).

As motivating examples, consider the following inconsistencies in the published literature:

— *Correctness.* CURE [50] identifies correct patches that pass all test cases and are semantically equivalent to the developer bug-fix (agreed after manual review by two of the authors), while Codex [25] relies on Leetcode public and private test cases. Each of these is reasonable, but their differences make it difficult to compare results.
— *Fault Localization.* Sequencer [27] assumes that the input includes which line of code needs to be changed (perfect fault localization); iFixR [57] uses human-generated bug reports to locate faults; and most earlier APR evaluations included fault localization as part of the task.
— *Termination.* Comparing tool efficiency is problematic because different tools use different stopping conditions: RewardRepair [133] terminates after generating and evaluating the best 200 patches per bug (regardless of the time taken); CURE [50] generates 10,000 candidate patches, validating the top 5,000; and Dlfix [69] stops when its fixed-length time budget is exhausted.

These are just a few examples of significant differences in how groups of authors report evaluation results on common benchmarks. In light of these and other issues, this article provides a topical review of recent developments in ML-based APR tasks. Several high-quality general surveys of APR research have been published [39, 62, 84]; here, we take a narrower view, examining all of the articles published at the top five software engineering venues since 2018, focusing particularly on those that use ML. This allows us to analyze emerging trends in venues favored by most APR researchers.

Methodologically, we analyzed Monperrus's The Living Review of Automated Program Repair (hal-01956501, version 5). As it is frequently updated, Monperrus's living review includes a diversity of APR topics, including repair domains and error types, ancillary improvement methods, and human studies. Using this review to seed our own, we determined the five venues publishing the largest volume of APR articles in The Living Review: **International Conference on Software Engineering** (**ICSE**), **Transactions on Software Engineering** (**TSE**), **Foundations of Software Engineering** (**FSE**), **Automated Software Engineering** (**ASE**), and **Empirical Software Engineering** (**EMSE**). We then independently reviewed articles published in each of the five fora from 1 January, 2018 through 31 December, 2023. We identified primary works for our study from article metadata using the primary keywords *patch* and *repair*. We next searched for other potentially related works via secondary keywords: *automate*, *fix*, *fault*, *bug*, *vulnerability*, and *generate*. Articles containing any of these terms (or their stems or variants) triggered a manual review of the abstract, which we examined for APR-related themes. Of the 118 articles identified by our methodology, only 65 are cross-listed in the Monperrus living review.

After a brief primer on commonly used terms (Section 2), we consider the rise of diverse ML approaches as applied to APR (Section 3), specific concerns with respect to how training and testing data interact (Section 4), and the role that benchmarks play in mediating and exposing these issues

(Section 5). We discuss a new APR competition and a recent proposal to standardize ML model application to APR tasks (Section 6), before concluding the article.

2 Background

2.1 Transformer Models and Attention

Initially developed for NLP, **neural machine translation** (**NMT**) models employ neural networks for machine translation tasks like sentence prediction and translating from one language to another [29]. Current NMT models are usually implemented as encoder-decoder transformer architectures with attention [76, 114], which improves on earlier LSTM-based [106] encoder-decoder architectures [29]. Transformer architectures vary: encoder [32], decoder [25], and encoder-decoder pairs [65, 98] are used in different circumstances. Encoder components extract information and embed it in a latent space, while decoders generate content based on the information in the input's representation in that latent space. Because encoder-only architectures have limited generative capabilities in zero-shot learning [109, 116], decoder-only and encoder-decoder models are often adopted for generative tasks. While not specific to transformers, *attention* is a major architectural feature that assists scaling by accommodating input sequences of varying lengths [22]. Attention allows the identification of input-output relationships when the position of those features is unknown. This is important for tasks like APR, where inputs (usually tokenized source code) vary widely in their structure and length.

2.2 Language Models

Commonly used in NLP, language models are used to learn contextual representations of text and are usually implemented as deep neural networks. A **pre-trained language model** (**PLM**) has parameters initialized from training on large corpora for generic text representation. They amortize much of the effort required to train on such large datasets and can be fine-tuned later for specific tasks [32, 109]. **Large language models** (**LLMs**) [138] are, as the name suggests, very large-scaled language models, usually with a billion or more parameters. LLMs inherit the pre-training and fine-tuning paradigms and architectures of PLMs. LLMs are seeing quick adoption in many fields, including APR, where their better performance at complex generative tasks is a relevant competitive advantage.

2.3 Data Bias

Data bias is a class of ML errors that occurs when features are not equitably represented in the dataset, leading to distorted results, compromised outcomes, and low accuracy [81]. Identifying data biases and their potential sources becomes increasingly difficult as the size of the training dataset increases. Unintended data bias is a common problem in natural language tasks when employing real-world data [12]. However, how bias may manifest in APR tasks is not well-studied. For example, an NPR model that uses Codex might be biased towards human-readable input-output pairs and be inaccurate when faced with algorithmically generated code, because Codex filters suspected auto-generated source code from its training corpora [25]. While this and other potential code-related data biases should be explored in future research, researchers often mitigate known data bias by supplementing the training data with synthetic data [47]. Synthetic data are generated such that relevant statistical properties are conserved, while other factors are controlled. The articles we reviewed did not explicitly discuss data bias or how to handle it.

2.4 Data Leakage

In ML, data leakage occurs when the training data includes information or features relevant to the prediction. This often leads to overestimates of a model's likely performance on new, unseen

Table 1. Frequently used Keywords for 118 APR Articles: Author-selected Keywords that Were Used four or More Times in the Dataset

keyword	count	keyword	count	keyword	count	keyword	count
program	101	generation	10	localization	6	quality	5
repair+	93	debugging*	10	based	6	✓networks	4
automated*	36	✓deep	10	test	6	search	4
✓learning	25	analysis	10	java	6	†assessment	4
software	25	testing	9	engineering	5	static	4
patch+	23	vulnerability*	9	bug*	5	mutation	4
automatic*	19	✓machine	9	symbolic	5	fixing*	4
code	15	fault*	7	†overfitting	5	apr	4
synthesis	12	empirical	7	†correctness	5	bugs*	4
✓neural	11	execution	7	fix*	5	representation	4

Keywords associated with ML are tagged with ✓and patch correctness with †. Similarly, primary keywords are tagged with +, secondary with *.

data [53]. While data leakage can occur when the prediction or a proxy for the prediction is present in the training data, other leakage and contamination occurs when individual cases contain similar content (*near-duplication* [8, 36, 58], Type II and Type III code clones [101]) or the same content (*duplication* [64], Type I code clones [101]). ML researchers generally try to minimize these issues when segmenting training from test data to avoid duplication, employing techniques for *de-duplication* and *decontamination*. However, as Section 4 discusses, these methods are often insufficient in the context of APR. In language models, duplication can occur at different levels of granularity or abstraction, i.e., sentences, documents, and datasets. Similarly, duplication can occur at varying levels in source code, i.e., statements, functions, classes, and files.

3 Diverse ML Approaches Pervade APR

Table 1 summarizes the author-chosen keywords covered by the 118 articles in our corpus. Of these, 29% list ML-related keywords. In 2018, this was relatively infrequent—fewer than one in six articles did so. But by 2023, more than three in every four articles published employed ML techniques. This is a striking change in a short period of time.

3.1 The Rise of LLMs

Increasing model size and data sizes of LLMs have led to a number of apparently emergent capabilities, including step-by-step reasoning and in-context learning [117, 118]. These properties have intrigued researchers [14] and the public with the potential of LLMs as a component of **artificial general intelligence** (**AGI**). But these properties, the factors that contribute to their emergence, and why they are absent in smaller models are not well understood [105]. Regardless, LLMs are becoming popular with APR researchers, usually for patch generation. Of the 118 articles included in our corpus, 61 propose new APR techniques. Of these 61 articles proposing new APR techniques, 22 used ML models, of which 7 were LLMs. Further, 6 of the 7 APR techniques that used LLMs were published in the past two years.

3.2 Code-Specific Language Models

Some PLMs and LLMs include source code in their training corpora [25, 30, 68, 104], often obtaining source code from public code repositories like GitHub or code-related help platforms like StackOverflow. These models generate more accurate results in reasoning tasks when those tasks

are formatted into code-like structures [78]. Specialized *code* LLMs, fine-tuned on source code, have shown promise as generative models for tasks including code completion [25] and program synthesis [7]. Preliminary work applies LLMs to the problem of APR [4, 34, 37, 50, 94, 122, 123, 131].

3.3 Architectures

Other specialized ML architectures have been applied to code-related tasks. When neural network techniques are used for APR, they are called ***neural program repair*** (**NPR**). While other code-related tasks have used structural architectures, like graph-based neural networks for code summarization [63], most NPR architectures adopt NMT models. NMT-based NPR models can use custom implementations [113], but code LLMs are increasingly popular in tools like AlphaRepair [123] (CodeBERT [35]), CoditT5 [137] and RewardRepair [133] (CodeT5 [98]), and SelfAPR [131] (PLBART [3]).

In our review, we found that the transformer architecture is used more often than any other ML architecture. Of the 29 articles that use ML techniques, we identified 20 that use the transformer model, including the following 18 tools: RewardRepair [133], CURE [50], DLFix [69], VRepair [26], SeqTrans [28], SynShine [4], SequenceR [27], AlphaRepair [123], VulRepair [37], Recoder [140], Quatrain [112], SelfAPR [131], TransRepair [67], Reptory [88], TENURE [83], Tare [141], Knod [49], and Rete [94]. Other neural network-based architectures are used as classifiers, like the BiLSTM-based multi-classifier used in TRANSFER [82], as generative models, the RNN-based SynFix [10] and ASTNN-based code representation model used in AccPR [127], and finally as a learning model like the CNN used to learn features of fix patterns [71]. The first wave of models to use ML for APR were largely NMT-based models, where the paradigm was to *translate* from buggy code to repaired code. The current trend is to adopt LLMs, where the paradigm is to delete a buggy line, and use generative text models to predict code to fill the resulting gap.

3.4 Attention Mechanisms

Attention is handled inconsistently in the transformer-based tools we considered: one does not mention a specific attention model [140], five describe it in great detail [27, 28, 49, 69, 141], while the remainder cite attention research like Vaswani [114] and Luong [76] [33, 50, 88, 94], give a brief explanation [26, 67, 83], or simply cite an inherited NMT model [4, 37, 111, 112, 123, 131, 133]. Although CURE [50] inherits its multi-headed attention architecture from its pre-trained language model GPT, it cites Vaswani's landmark article [114] for attention. In contrast, Seqtrans [28] describes attention in great detail while reusing OpenNMT which leverages Luong's attention mechanism [76]. This variability of detail may result from the lack of accepted attention taxonomies [22].

4 Data Leakage Contaminates Models and Benchmarks

4.1 Data Leakage

Data leakage commonly occurs when training data includes information relevant to the task that it should not be able to access. When the reviewed literature touched on data leakage, *train-test segmentation* was common, i.e., the practice of distinctly separating the training corpus from the testing set such that coincident content is minimized. SelfAPR [131], which focuses on project-specific knowledge through self-supervised learning, splits its training and testing dataset along temporal boundaries, i.e., the timestamp of the bug-repairing commit in a project. This approach is also used by CURE [50], which trains on data before 2006 in order to avoid contamination with the benchmarks (dating from 2007 onward). Overfitting Detection System [130], a tool for correctness assessment of APR patches, adopts project-based benchmark segmentation to distinguish its training set (Bugs.jar [102], Bears [79], and Defects4J [52]) from testing by excluding

from consideration any project (e.g., the Apache Math library) that exists in more than one of these sources, and therefore might foil standard random-sample segmentation due to duplicates. For its syntactic training, RewardRepair [133] leverages existing training corpora composed of bug-fix pairs and does not perform any train-test segmentation or decontamination for this content. Synshine [4], which uses a pre-trained RoBERTa transformer model, performed standard segmentation on its dataset consisting of 1.7 M pairs of erroneous and fixed programs, selecting a random test set of 100 K for evaluation. We recommend increased scrutiny of data leakage in evaluations of ML-based APR, particularly when evaluation is carried out using benchmarks that were not designed with the needs of ML model evaluation in mind. This is particularly true for techniques that use pre-trained models, because bugs from APR datasets are many years old and are widely available on the public internet, suggesting that PLMs have likely scraped them.

4.2 Contamination

Contamination poses a significant risk to the validity of ML-based APR results, since it can lead to reported performance levels that fail to materialize in practice. Duplicated data lead to bias in the trained models towards particular examples, which increases the likelihood that generated content is memorized [19]. Our review did not find APR articles addressing the issue of near duplicates, although one did discuss potential contamination of PLMs [122]. Decontamination can improve the performance of LLMs [17, 30], but it is challenging. First, since pre-trained weights are inherited, de-duplication of the training set by end-users is not possible. Second, because details of training may not be available to the user, including the training corpus itself, decontamination of test data by outside users can be impossible.

Five of the articles that use LLMs for APR do not consider the impact of data leakage and contamination [4, 34, 37, 94, 123]. Xia, et al. [122] reports contamination information for each evaluated PLM with open-source training. They enumerate the repairs (identical to the developer provided change) where the modified code was reproduced from the training data. SelfAPR [131], which opts for training its own LLM, removes all duplicate buggy samples from the training corpora. CURE [50], which also retrains its LLM, populates its corpora with GitHub open-source Java projects rolled back before 2006, then manually removes duplicate bugs from their datasets, a coarse deduplication effort using the same methodology as CoCoNut [77]. CURE's decontamination policy assumes independence among GitHub projects and between code commits and repairs (a strong assumption), where SelfAPR assesses each and retains only unique samples. Three LLM methods for APR addressed data leakage by deduplicating their corpora. One removed duplicates using syntax [59], another considered semantics [112], and the third did not specify a deduplication method [111]. Several methods have been developed for decontaminating datasets, discussed below.

4.3 Decontaminating Benchmarks

Even when decontamination is an explicit goal, the capacity to accomplish it—or even to quantify the level of contamination—is often unavailable. A standard decontamination method has not yet emerged for pre-trained models in language task settings, much less code task settings. For example, GPT-3 used N-grams to identify contamination, where a *gram* is a lowercase, whitespace delimited word without punctuation [13]. In comparison, GPT-4 uses multi-sample substring matching as a filter [92]. Because its training corpora are unavailable, directly applying OpenAI's GPT decontamination policy to new benchmarks is infeasible for third parties using their models. Additionally, standard approaches for estimating the likelihood of data contamination in training after the fact, such as quantifying the entropy gap between seen and unseen data [107], are not applicable when the model itself is not accessible (e.g., because users have API access). *Machine*

unlearning updates a pre-trained model's existing weights, retraining or approximating the effects of removing implicated data [15, 16, 121, 136]. Primarily focused on security and privacy concerns, research in machine unlearning is gaining momentum, particularly with the 2023 NeurIPS challenge announcement [1].

4.4 Memorization and Generalization

Memorization, a type of data leakage, is defined as the propensity of language models to generate sequences from training data verbatim, without capturing their meaning [18]. Memorization may be beneficial for some language tasks that reproduce learned facts, but its benefit for code tasks is mixed. It may help to repair bugs that are duplicates of what has been seen, but memorization is problematic when a model must generalize. While the memorization of specific training examples is difficult to prevent, recent research shows that removing duplicates (Type I code clones [101]) or near-duplicates (Type II and Type III code clones [101]) in the training corpora reduces verbatim memorization [64].

Few language models outline a decontamination policy and even fewer provide a decontamination package with their model that detects contaminated test samples and produces a clean version of a benchmark, e.g., EleutherAI [96]. Related work suggests two paths forward for tackling this issue. First, testing approaches from language model security- and privacy-focused research can be leveraged to determine the likelihood of a test sample's existence in the dataset [19, 95]. One such method is the use of canary strings [18]: easily detectable synthetic data that are obvious when reproduced. Second, taking cues from adversarial text generation [38] and Seqtrans [28], variable names and user-defined type names can be perturbed during validation, when a model is evaluated for generalization.

4.5 Granularity

While document-, statement-, and sentence-level duplication is better studied in language models for non-code tasks, most de-duplication occurs at the file level for code models [6, 124]. This may leave duplicate code excerpts intact, when duplication occurs at other levels of abstraction, e.g., for duplicated functions. As a simple example, consider multiple projects that reference a solution for a problem from StackOverflow, resulting in the adoption and adaptation of similar functions. While lower-level segmentation may not be appropriate for code tasks like summarization, current NPR strategies that perform translation on small code fragments may be particularly susceptible to duplication. NPR techniques use various input granularities, ranging from max-token-limited context [28, 69, 131, 133] to the entire buggy method [50], which determine the granularity of each model's semantic features. To address these issues, cross-contamination should be measured and reported between the evaluation and training datasets. Similarity measures could be adapted from clone detection methods [100] and reflect the granularity of semantic features in the code [51].

4.6 Impact of Near-duplicates on Code Tasks

Existing methods for deduplication often search for and remove exact matches. This is insufficient for ML-based APR, especially in the context of emerging LLMs. Near-duplicate code is a significant threat to language models [75]. When near-duplicate code is included in training and benchmark datasets, performance metrics can inflate up to 100% in comparison to de-duplicated corpora [6]. Language model contamination policies have leveraged metrics that measure similarities for natural language, rather than those used for code. Luckily, the problem is amenable to text-based clone detection and code similarity metrics, which is often used to detect plagiarism [99] or license violations [44]. While some string matching techniques can outperform specialized tools for code with

heavy structural changes, highly specialized source code similarity detection techniques perform better than general textual measures of string similarity [99].

5 Benchmarks do Not Enforce Standardized Evaluation

5.1 Performance Reporting

Consider how results are reported for experimental evaluations using the ManyBugs benchmark [60], an older collection of large, open-source C programs, compared to how they are reported for Defects4J [52], a more recent collection of small, open-source Java libraries. For articles that report experiments on ManyBugs, authors usually report [80] detailed run-time results in addition to the success rate of repairing defects. However, articles evaluated on Defects4J frequently omit this information, focusing only on whether or not a candidate patch (or successful repair) was found. When it is included, the reported information varies widely and can include an upper-bound on wall clock time [72] or setting a limit on number of test suite evaluations [50], without reporting efficiency in sufficient detail to allow comparison across articles.

5.2 Fault Localization

An algorithm that has to handle fault localization is facing a different, harder problem, compared to an algorithm that does not. Consider the distinction between *patch generation* (the task of generating code modifications to be evaluated) and full *automated repair* (an end-to-end process which includes the sub-problems of fault localization and a priority scheme when multiple solutions are proposed). Some recent articles evaluate performance with and without assuming perfect fault localization [34, 83, 123, 140, 141] and clearly specify which approach they have taken. But performance reporting is not universal, which suggests that care must be taken when comparing results from different projects.

Some authors exclusively use information from running the test cases and tracing execution to inform a spectrum-based fault localization algorithm that predicts where the defect is likely to be [2, 5, 9, 23, 41–43, 45, 46, 55, 56, 61, 66, 69, 87, 91, 93, 103, 119, 125, 131, 133]. Others take the location where the human applied the repair (provided by the benchmark for validating a solution) as the input directly, and focus attention on or around that location, excluding the problem of fault localization [26–28, 33, 34, 37, 49, 50, 83, 88, 94, 122, 123, 127, 140, 141]. NPR tools usually assume perfect fault localization, relying on the source file names and corresponding line numbers involved in the developer's repair to scrape the fault's location, rather than estimating it as part of the repair process [139]. Between these extremes are a variety of choices, many of which have been adopted by APR algorithm designers [80].

5.3 Correctness

Many articles use a proxy for *correctness*, most often comparing the generated patch to a single reference repair supplied by a human developer [2, 23, 27, 40–42, 46, 48–50, 54, 56, 69, 73, 82, 85, 88, 93, 94, 103, 119, 122, 125, 126, 129, 131, 133, 139, 141]. This restricts the solutions that are judged acceptable to those that resemble the choices human developers happened to make, whether or not that is the only way (or even the best way) to satisfy the program's required functionality. Since there are an infinite number of ways to implement any required functionality, defining correctness relative only to an arbitrary human example is extraordinarily strict.

5.4 APR Benchmarks Differ from ML Datasets

Because of their different intended uses and construction requirements, APR benchmarks [52, 60, 70, 79, 102] are too small to train ML tools. They consist of a limited number of bug/fix pairs

with developer-written patches and test cases to define constraints on expected program behavior. The limited number of samples makes it difficult for ML models to be trained without other sources [133]. ML datasets, in order to be large enough, cannot be curated to the same extent that APR benchmarks are. For example, the CrossVul dataset [89] is composed of differences between the commit for a bug fix and the previous program version. However, these datasets do not include replicable build environments for patch evaluation, currently deemed unnecessary in ML patch generation [97]. Other large datasets of bug-fix pairs [50] similarly lack execution environments like test suites, because they are essentially program-text-only.

5.5 Access to source code, bug information, and repair information

APR tools have requirements that interact with benchmark design. For example, some APR tools require program-specific historical development information, using **information retrieval** (**IR**) methods that analyze a program's commit history or commit messages. From our review, only iFixR [57] uses a bug-report based IR method. These IR methods are supported in most benchmarks, since they provide links to each program's public code repository. Full containment of this history within the benchmark is often infeasible. Benchmarks that focus on small or novice-generated programs, like CodeFlaws [110] and IntroClass [60], are exceptions, because the histories are short enough to be included.

5.6 Reproducible program builds

Earlier APR tools rely on the ability to build a program from source code; however, until recently [97], ML-based tools for APR have avoided reproducing builds. Even with containerization, there can be conflicting requirements or dependencies that make building or running some APR tools incompatible with a buggy program's container. This is particularly challenging for tools that augment build processes with their own requirements like Angelix [80] and Prophet [74]. In comparison to C and C++ builds, Gradle and Maven are widely accepted build tools for Java programs that tightly control a program's external dependencies. These standard build tools and the stability of Java and JRE make Java programs amenable for APR research.

From our review, we can see the impact of build overhead in the number of articles choosing to evaluate against ManyBugs [60], an early C benchmark with incomplete build support, versus Defects4J [52], a Java benchmark complete with Maven build infrastructure. There are five articles in our sample that contain data from evaluating ManyBugs [2, 20, 87, 90, 93]; however only two present new evaluation results, with the rest summarizing data of experiments from years past. In contrast there are 44 articles in our sample that contain data from evaluating Defects4J [5, 9, 20, 23, 24, 27, 40, 41, 45, 46, 48–50, 55–57, 59, 69, 71, 73, 82, 83, 85–87, 103, 111, 112, 115, 119, 120, 122, 123, 125, 126, 128–134, 140, 141]. While there are other factors that influence the differences in adoption patterns for these two datasets, e.g., the fact that they target different languages, some authors [77] have been unable to execute ManyBugs and so performed syntactic comparison to the human-supplied repair as a proxy for evaluation.

While obviously at a disadvantage due to this build complexity, C and C++ programs account for a significant fraction of software in development [11]. This prevalence, together with the emergence of code LLMs for multiple languages [21], indicates that APR benchmarks would benefit from a consolidated build and run environment that enables a variety of APR tools and language bases.

5.7 Consistent reporting measures

In ML, particularly for NLP tasks, beam search [108] determines how narrow or broad a search is, which affects its computational cost. This is controlled by the hyperparameter *beam size k*. On

each evaluative step, the k sequences with the largest output weights from the set of remaining sequences are selected. A small beam size collapses toward simple greedy search, and a large beam size extends toward exhaustive search: a classic tradeoff of runtime vs. quality. We found that NPR techniques have not converged on a standard metric for search efficiency and generate a wide number of unverified patches for each buggy program. We found hints that beam size is trending as a standard, with justification of beam size by reference to other evaluations [26, 49]. Beam sizes ranged anywhere from 50 [26, 28] to 200 [133] to 1000 [49, 94]. However, beam size is an ineffective measure of search efficiency as it neglects to account for the model's generative vocabulary size, $|Y|$, and sequence length. Unfortunately, we found that generative vocabulary size was not always reported [49, 69], making search efficiency comparisons difficult. When vocabulary is accounted for, NPR APR models have widely varying effective search sizes: SeqTrans ($k = [1...50]$, $|Y| = 8,000$); VRepair ($k = 50$, $|Y| = 5,000$); RewardRepair($k = [30, 100, 200, 500]$, $|Y| = 32,128$)); CURE($k = [1000]$, $|Y| = 50,000$).

6 Discussion

6.1 Standup4NPR

StandUp4NPR [139] was introduced to standardize the evaluation of ML-based APR tools. It has a well-defined interface for training and testing, and can impose resource restrictions on all models to facilitate accurate comparison between tools. The training and testing data are taken from existing datasets and benchmarks, and are then subsequently processed and filtered, in an attempt to minimize the incidence of exact duplicates.

Many aspects of this project are promising: standardizing data access, resource usage, runtime, and evaluation criteria across a variety of models creates a much more level field for comparison. However, other design features of StandUp4NPR can pose problems. By restricting data access to the training examples provided, what is being tested is how efficient and effective an architecture is at leveraging the data in the training dataset. This rules out other tasks that could improve APR efficiency of ML tools: careful curation of other, independent datasets; using auxiliary inputs (like program history, or the history from other projects) not available within the benchmark, and so on.

A related issue is that, because StandUp4NPR was formulated for the evaluation of neural techniques, it is at times ill-designed for use by traditional APR algorithms. GenProg [61] and ARJA [135], for example, use information about how a mutation has performed to guide the search at the next time-step. But the StandUp4NPR framework assumes that all patches are generated simultaneously and then evaluated independently. This lack of an evaluation feedback loop means that many search-based techniques cannot be compared under this framework.

6.2 APR-COMP 2024

Recently, the 1st International Competition on Automated Program Repair was announced. It has four competition tracks: functional errors, vulnerabilities, student assignments, and AI-generated code. For the functional repair track, benchmark programs are sampled from existing benchmarks [52, 60, 79, 102]. This means that it will inherit the issues that these benchmarks have, and that the content is knowable to model designers in advance. This differs from how some annual ML competitions, e.g., ImageNet [31], are structured; they released a public training dataset, but held in private the test data. That said, having a standard API for access to multiple benchmarks is a large benefit for the interoperability of repair tools. Also, as with StandUp4NPR, common limits on evaluation time and resource usage facilitate comparison of the resulting data. Finally, because submission is via public pull request, it is in practice requiring disclosure of an executable object, which not all APR articles contain.

7 Conclusion

APR is a vibrant and fast-evolving field within software engineering. There is already economic benefit from the application and deployment of APR techniques, evidenced by its increasing uptake by industry. However, as it has matured, the field has concentrated evaluation effort around a few popular benchmarks, which are now showing their age. In particular, the rise of ML approaches in APR poses problems for the evaluation of existing benchmarks.

In this article, we have detailed the origin of some of these issues and have suggested how we might translate insights from other fields to tackle the challenges we have in common. Data leakage is a significant threat to the validity of results, particularly language models whose training corpora are unavailable or undisclosed. The wide variety in the reporting of results additionally makes comparing across evaluations challenging at best and infeasible at worst. Due to these inherent issues with the methods used to evaluate the success of APR tools, more and larger empirical studies and comparisons will not shed light until these problems are addressed.

Acknowledgments

The authors gratefully acknowledge the partial support of NSF (CCF CCF2211750, CCF2211749), DARPA (N6600120C4020), ARPA-H (SP4701-23-C-0074), and the Santa Fe Institute.

References

[1] NeurIPS. 2023. Machine Unlearning Challenge. Retrieved September 11, 2023 from https://unlearning-challenge.github.io/

[2] Afsoon Afzal, Manish Motwani, Kathryn T. Stolee, Yuriy Brun, and Claire Le Goues. 2021. SOSRepair: Expressive semantic search for real-world program repair. *IEEE Transactions on Software Engineering* 47, 10 (2021), 2162–2181.

[3] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the NAACL-HLT 2021, Online, June 6-11, 2021*. Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.), Association for Computational Linguistics, 2655–2668.

[4] Toufique Ahmed, Noah Rose Ledesma, and Premkumar Devanbu. 2023. SynShine: Improved fixing of syntax errors. *IEEE TSE* 49, 4 (2023), 2169–2181.

[5] Aldeida Aleti and Matias Martinez. 2021. E-APR: Mapping the effectiveness of automated program repair techniques. *Empirical Software Engineering* 26, 5 (2021), 99:1–99:30.

[6] Miltiadis Allamanis. 2019. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the New Ideas, New Paradigms, and Reflections on Programming and Software* (Athens, Greece). ACM, New York, NY, USA, 143—153.

[7] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language Models. arXiv:2108.07732 [cs]

[8] Björn Barz and Joachim Denzler. 2020. Do we train on test data? Purging CIFAR of near-duplicates. *Journal of Imaging* 6, 6 (2020), 41.

[9] Samuel Benton, Xia Li, Yiling Lou, and Lingming Zhang. 2020. On the effectiveness of unified debugging: An extensive study on 16 program repair systems. In *Proceedings of the ASE 2020, Melbourne, Australia, September 21-25, 2020*. IEEE, 907–918.

[10] Sahil Bhatia, Pushmeet Kohli, and Rishabh Singh. 2018. Neuro-symbolic program corrector for introductory programming assignments. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg Sweden, 60–70.

[11] Tegawende F. Bissyande, Ferdian Thung, David Lo, Lingxiao Jiang, and Laurent Reveillere. 2013. Popularity, interoperability, and impact of programming languages in 100,000 open source projects. In *Proceedings of the Annual Computer Software and Applications Conference*. IEEE, Kyoto, Japan, 303–312.

[12] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2019. Nuanced metrics for measuring unintended bias with real data for text classification. In *Companion Proceedings of The 2019 World Wide Web Conference*. ACM, San Francisco USA, 491–500.

[13] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, and Tom

Henighan. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33. NIPS.

[14] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early Experiments with GPT-4. arXiv:2303.12712 [cs]

[15] Yinzhi Cao and Junfeng Yang. 2015. Towards making systems forget with machine unlearning. In *Proceedings of the Security and Privacy*. IEEE, San Jose, CA, 463–480.

[16] Yinzhi Cao, Alexander Fangxiao Yu, Andrew Aday, Eric Stahl, Jon Merwine, and Junfeng Yang. 2018. Efficient repair of polluted machine learning systems via causal unlearning. In *Proceedings of the Asia Conference on Computer and Communications Security*. ACM, Incheon Republic of Korea, 735–747.

[17] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. 2023. Quantifying memorization across neural language models. In *Proceedings of the 11th International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

[18] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *Proceedings of the USENIX Security*. USENIX.

[19] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting training data from large language models. In *Proceedings of the USENIX Security*. 2633–2650.

[20] Padraic Cashin, Carianne Martinez, Westley Weimer, and Stephanie Forrest. 2019. Understanding automatically-generated patches through symbolic invariant differences. In *Proceedings of the Automated Software Engineering*. IEEE, 411–414.

[21] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2023. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Trans. Software Eng.* 49, 7 (2023), 3675–3691.

[22] Sneha Chaudhari, Varun Mithal, Gungor Polatkan, and Rohan Ramanath. 2021. An attentive survey of attention models. *ACM Transactions on Intelligent Systems and Technology* 12, 5 (2021), 1–32.

[23] Liushan Chen, Yu Pei, and Carlo A. Furia. 2021. Contract-based program repair without the contracts: An extended study. *IEEE Transactions on Software Engineering* 47, 12 (2021), 2841–2857.

[24] Liushan Chen, Yu Pei, Minxue Pan, Tian Zhang, Qixin Wang, and Carlo A. Furia. 2023. Program repair with repeated learning. *IEEE Transactions on Software Engineering* 49, 2 (2023), 831–848.

[25] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs]

[26] Zimin Chen, Steve Kommrusch, and Martin Monperrus. 2023. Neural transfer learning for repairing security vulnerabilities in c code. *IEEE Transactions on Software Engineering* 49, 1 (2023), 147–165.

[27] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noel Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. SEQUENCER: Sequence-to-sequence learning for end-to-end program repair. *Transactions on Software Engineering* 47, 9 (2019), 1943–1959.

[28] Jianlei Chi, Yu Qu, Ting Liu, Qinghua Zheng, and Heng Yin. 2023. SeqTrans: Automatic vulnerability fix via sequence to sequence learning. *IEEE Transactions on Software Engineering* 49, 2 (2023), 564–585.

[29] Kyunghyun Cho, Bart van Merrienboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the Properties of Neural Machine Translation: Encoder-Decoder Approaches. In *Syntax, Semantics and Structure in Statistical Translation*. Association for Computational Linguistics, Doha, 103–111.

[30] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana

Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, Noah Fiedel, and Ruslan Salakhutdinov. 2023. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[31] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition*. IEEE, Maimi, 248–255.

[32] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1*. Jill Burstein, Christy Doran, and Thamar Solorio (Eds.), Association for Computational Linguistics, 4171–4186.

[33] Yangruibo Ding, Baishakhi Ray, Premkumar Devanbu, and Vincent J. Hellendoorn. 2020. Patching as translation: The data and the metaphor. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Virtual Event.

[34] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *Proceedings of the International Conference on Software Engineering*. IEEE, 1469–1481.

[35] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1536–1547.

[36] Maik Fröbe, Janek Bevendorff, Jan Heinrich Reimer, Martin Potthast, and Matthias Hagen. 2020. Sampling bias due to near-duplicates in learning to rank. In *Proceedings of the SIGIR 2020*. ACM, Virtual Event China, 1997–2000.

[37] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *Proceedings of the ESEC/FSE*. ACM, Singapore Singapore, 935–947.

[38] Ji Gao, Jack Lanchantin, Mary Lou Soffa, and Yanjun Qi. 2018. Black-box generation of adversarial text sequences to evade deep learning classifiers. In *Proceedings of the 2018 IEEE Security and Privacy Workshops*. IEEE, 50–56.

[39] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic software repair: A survey. *TSE* 45, 1 (2019), 34–67.

[40] Ali Ghanbari. 2019. Toward practical automatic program repair. In *Proceedings of the ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 1262–1264.

[41] Ali Ghanbari and Lingming Zhang. 2019. PraPR: Practical program repair via bytecode mutation. In *Proceedings of the ASE 2019, San Diego, CA, USA, November 11-15, 2019*. IEEE, 1118–1121.

[42] Davide Ginelli, Matias Martinez, Leonardo Mariani, and Martin Monperrus. 2022. A comprehensive study of code-removal patches in automated program repair. *Empir. Softw. Eng.* 27, 4 (2022), 97.

[43] Matthías Páll Gissurarson, Leonhard Applis, Annibale Panichella, Arie Van Deursen, and David Sands. 2022. PropR: Property-based automatic program repair. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, Pittsburgh Pennsylvania, 1768–1780.

[44] Yaroslav Golubev, Maria Eliseeva, Nikita Povarov, and Timofey Bryksin. 2020. A study of potential code borrowing and license violations in java projects on github. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 54–64.

[45] Jinru Hua, Mengshi Zhang, Kaiyuan Wang, and Sarfraz Khurshid. 2018. SketchFix: A tool for automated program repair approach using lazy candidate generation. In *Proceedings of the ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04-09, 2018*. Gary T. Leavens, Alessandro Garcia, and Corina S. Pasareanu (Eds.), ACM, 888–891.

[46] Jinru Hua, Mengshi Zhang, Kaiyuan Wang, and Sarfraz Khurshid. 2018. Towards practical program repair with on-demand candidate generation. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg Sweden, 12–23.

[47] Nikita Jaipuria, Xianling Zhang, Rohan Bhasin, Mayar Arafa, Punarjay Chakravarty, Shubham Shrivastava, Sagar Manglani, and Vidya N. Murali. 2020. Deflating dataset bias using synthetic data augmentation. In *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, Seattle, WA, USA, 3344–3353.

[48] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1430–1442.

[49] Nan Jiang, Thibaud Lutellier, Yiling Lou, Lin Tan, Dan Goldwasser, and Xiangyu Zhang. 2023. KNOD: Domain knowledge distilled tree decoder for automated program repair. In *Proceedings of the ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1251–1263.

[50] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-aware neural machine translation for automatic program repair. In *Proceedings of the International Conference on Software Engineering*. IEEE, Madrid, Spain, 1161–1173.
[51] Yuan Jiang, Xiaohong Su, Christoph Treude, and Tiantian Wang. 2022. Hierarchical semantic-aware neural code representation. *Journal of Systems and Software* 191 (2022), 111355.
[52] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the International Symposium on Software Testing and Analysis*. ACM, San Jose, CA, USA, 437–440.
[53] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6, 4 (2012), 1–21.
[54] Maria Kechagia, Sergey Mechtaev, Federica Sarro, and Mark Harman. 2022. Evaluating automatic program repair capabilities to repair API misuses. *IEEE Transactions on Software Engineering* 48, 7 (2022), 2658–2679.
[55] Jindae Kim and Sunghun Kim. 2019. Automatic patch generation with context-based change application. *Empir. Softw. Eng.* 24, 6 (2019), 4071–4106.
[56] Anil Koyuncu, Kui Liu, Tegawendé F. Bissyandé, Dongsun Kim, Jacques Klein, Martin Monperrus, and Yves Le Traon. 2020. FixMiner: Mining relevant fix patterns for automated program repair. *Empir. Softw. Eng.* 25, 3 (2020), 1980–2024.
[57] Anil Koyuncu, Kui Liu, Tegawendé F. Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. 2019. iFixR: Bug report driven program repair. In *Proceedings of the ESEC/FSE*. ACM, Tallinn Estonia, 314–325.
[58] Rayson Laroca, Valter Estevam, Alceu S. Britto, Rodrigo Minetto, and David Menotti. 2023. Do we train on test data? The impact of near-duplicates on license plate recognition. In *Proceedings of the 2023 International Joint Conference on Neural Networks*. IEEE, Gold Coast, Australia, 1–8.
[59] Thanh Le-Cong, Duc-Minh Luong, Xuan Bach D. Le, David Lo, Nhat-Hoa Tran, Bui Quang-Huy, and Quyet-Thang Huynh. 2023. Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning. *IEEE Transactions on Software Engineering* 49, 6 (2023), 1–20.
[60] Claire Le Goues, Neal Holtschulte, Edward K. Smith, Yuriy Brun, Premkumar Devanbu, Stephanie Forrest, and Westley Weimer. 2015. The manybugs and introclass benchmarks for automated repair of c programs. *Transactions on Software Engineering* 41, 12 (2015), 1236–1256.
[61] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A generic method for automatic software repair. *Transactions on Software Engineering* 38, 1 (2012), 54–72.
[62] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Communications of the ACM* 62, 12 (2019), 56–65.
[63] Alexander LeClair, Sakib Haque, Lingfei Wu, and Collin McMillan. 2020. Improved code summarization via a graph neural network. In *Proceedings of the 28th International Conference on Program Comprehension*. ACM, Seoul Republic of Korea, 184–195.
[64] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating training data makes language models better. In *Proceedings of the ACL 2022, Dublin, Ireland, May 22-27, 2022*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), ACL, 8424–8445.
[65] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the ACL 2020*. ACL, Online, 7871–7880.
[66] Leping Li, Hui Liu, Kejun Li, Yanjie Jiang, and Rui Sun. 2023. Generating concise patches for newly released programming assignments. *IEEE Trans. Software Eng.* 49, 1 (2023), 450–467.
[67] Xueyang Li, Shangqing Liu, Ruitao Feng, Guozhu Meng, Xiaofei Xie, Kai Chen, and Yang Liu. 2022. TransRepair: Context-aware program repair for compilation errors. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Rochester MI USA, 1–13.
[68] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien De Masson d.Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando De Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
[69] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. DLFix: Context-based code transformation learning for automated program repair. In *Proceedings of the International Conference on Software Engineering*. ACM, Seoul, South Korea, 602–614.
[70] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings of the Systems, Programming, Languages, and Applications: Software for Humanity*. ACM, 55–56.
[71] Kui Liu, Dongsun Kim, Tegawende F. Bissyande, Shin Yoo, and Yves Le Traon. 2021. Mining fix patterns for findbugs violations. *IEEE Transactions on Software Engineering* 47, 1 (2021), 165–188.

[72] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. TBar: Revisiting template-based automated program repair. In *Proceedings of the International Symposium on Software Testing and Analysis*. ACM, Beijing, China, 31–42.

[73] Kui Liu, Shangwen Wang, Anil Koyuncu, Kisub Kim, Tegawendé F. Bissyandé, Dongsun Kim, Peng Wu, Jacques Klein, Xiaoguang Mao, and Yves Le Traon. 2020. On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs. In *Proceedings of the ICSE*. ACM, Seoul South Korea, 615–627.

[74] Fan Long and Martin Rinard. 2015. *Prophet: Automatic Patch Generation via Learning from Successful Patches*. Technical Report MIT-CSAIL-TR-2015-027. CSAIL, Massachusetts Institute of Technology. 13 pages.

[75] Cristina V. Lopes, Petr Maj, Pedro Martins, Vaibhav Saini, Di Yang, Jakub Zitny, Hitesh Sajnani, and Jan Vitek. 2017. DéjàVu: A map of code duplicates on GitHub. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–28.

[76] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. 2015. Effective Approaches to Attention-based Neural Machine Translation. arXiv:1508.04025 [cs]

[77] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the International Symposium on Software Testing and Analysis*. ACM, Virtual Event, 101–114.

[78] Aman Madaan, Shuyan Zhou, Uri Alon, Yiming Yang, and Graham Neubig. 2022. Language models of code are few-shot commonsense learners. In *Proceedings of the EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*. Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.), Association for Computational Linguistics, 1384–1403.

[79] Fernanda Madeiral, Simon Urli, Marcelo Maia, and Martin Monperrus. 2019. Bears: An extensible java bug benchmark for automatic program repair studies. In *Proceedings of the 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering*. 468–478. arXiv:1901.06024 [cs]

[80] Sergey Mechtaev, Jooyong Yi, and Abhik Roychoudhury. 2016. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *Proceedings of the International Conference on Software Engineering*. ACM, Austin, TX, USA, 691–701.

[81] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. 2022. A survey on bias and fairness in machine learning. *Computing Surveys* 54, 6 (2022), 1–35.

[82] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, and Xudong Liu. 2022. Improving fault localization and program repair with deep semantic features and transferred knowledge. In *Proceedings of the ICSE*. ACM, Pittsburgh Pennsylvania, 1169–1180.

[83] Xiangxin Meng, Xu Wang, Hongyu Zhang, Hailong Sun, Xudong Liu, and Chunming Hu. 2023. Template-based neural program repair. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, Melbourne, Australia, 1456–1468.

[84] Martin Monperrus. 2018. *The Living Review on Automated Program Repair*. Technical Report. KTH Royal Institute of Technology.

[85] Manish Motwani and Yuriy Brun. 2023. Better automatic program repair by using bug reports and tests together. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering, Melbourne, Australia, May 14-20, 2023*. IEEE, 1225–1237.

[86] Manish Motwani, Sandhya Sankaranarayanan, René Just, and Yuriy Brun. 2018. Do automated program repair techniques repair hard and important bugs? *Empir. Softw. Eng.* 23, 5 (2018), 2901–2947.

[87] Manish Motwani, Mauricio Soto, Yuriy Brun, Rene Just, and Claire Le Goues. 2022. Quality of automated program repair on real-world defects. *IEEE Transactions on Software Engineering* 48, 2 (2022), 637–661.

[88] Marjane Namavar, Noor Nashid, and Ali Mesbah. 2022. A controlled experiment of different code representations for learning-based program repair. *Empirical Software Engineering* 27, 7 (2022), 190.

[89] Georgios Nikitopoulos, Konstantina Dritsa, Panos Louridas, and Dimitris Mitropoulos. 2021. CrossVul: A cross-language vulnerability dataset with commit data. In *Proceedings of the ESEC/FSE*. 1565–1569.

[90] Yannic Noller, Ridwan Shariffdeen, Xiang Gao, and Abhik Roychoudhury. 2022. Trust enhancement issues in program repair. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, Pittsburgh Pennsylvania, 2228–2240.

[91] Wonseok Oh and Hakjoo Oh. 2022. PyTER: Effective program repair for python type errors. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*. Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.), ACM, 922–934.

[92] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[93] Nikhil Parasaram, Earl T. Barr, and Sergey Mechtaev. 2021. Trident: Controlling side effects in automated program repair. *Transactions on Software Engineering* 48, 12 (2021), 4717–4732.

[94] Nikhil Parasaram, Earl T. Barr, and Sergey Mechtaev. 2023. Rete: Learning namespace representation for program repair. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, Melbourne, Australia, 1264–1276.

[95] Charith Peris, Christophe Dupuy, Jimit Majmudar, Rahil Parikh, Sami Smaili, Richard Zemel, and Rahul Gupta. 2023. Privacy in the time of language models. In *Proceedings of the 16th ACM International Conference on Web Search and Data Mining*. 1291–1292.

[96] Jason Phang, Herbie Bradley, Leo Gao, Louis Castricato, and Stella Biderman. 2022. EleutherAI: Going beyond" open science" to" science in the open". *arXiv preprint arXiv:2210.06413* (2022).

[97] Julian Aron Prenner and Romain Robbes. 2023. RunBugRun – An Executable Dataset for Automated Program Repair. arXiv:2304.01102 [cs.SE]

[98] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.

[99] Chaiyong Ragkhitwetsagul, Jens Krinke, and David Clark. 2018. A comparison of code similarity analysers. *Empirical Software Engineering* 23, 4 (2018), 2464–2519. DOI: https://doi.org/10.1007/s10664-017-9564-7

[100] Dhavleesh Rattan, Rajesh Bhatia, and Maninder Singh. 2013. Software clone detection: A systematic review. *Information and Software Technology* 55, 7 (2013), 1165–1199.

[101] Chanchal Kumar Roy and James R. Cordy. 2007. A survey on software clone detection research. *Queen's School of Computing TR* 541, 115 (2007), 64–68.

[102] Ripon K. Saha, Yingjun Lyu, Wing Lam, Hiroaki Yoshida, and Mukul R. Prasad. 2018. Bugs.Jar: A large-scale, diverse dataset of real-world java bugs. In *Proceedings of the 15th International Conference on Mining Software Repositories - MSR '18*. ACM Press, Gothenburg, Sweden, 10–13.

[103] Seemanta Saha, Ripon K. Saha, and Mukul R. Prasad. 2019. Harnessing evolution for multi-hunk program repair. In *Proceedings of the International Conference on Software Engineering*. IEEE, Montreal, QC, Canada, 13–24.

[104] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, et al. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv:2211.05100 [cs]

[105] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are Emergent Abilities of Large Language Models a Mirage? arXiv:2304.15004. Retrieved from https://arxiv.org/abs/2304.15004

[106] M. Schuster and K.K. Paliwal. Nov./1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing* 45, 11 (1997), 2673–2681.

[107] Virat Shejwalkar and Amir Houmansadr. 2021. Membership privacy for machine learning models through knowledge transfer. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 11 (2021), 9549–9557.

[108] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems* 27 (2014).

[109] Alexandre Tamborrino, Nicola Pellicano, Baptiste Pannier, Pascal Voitot, and Louise Naudin. 2020. Pre-Training Is (Almost) All You Need: An Application to Commonsense Reasoning.In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics*.

[110] Shin Hwei Tan, Jooyong Yi, Sergey Mechtaev, and Abhik Roychoudhury. 2017. Codeflaws: A programming competition benchmark for evaluating automated program repair tools. In *Proceedings of the 39th International Conference on Software Engineering Companion*. IEEE Press, 180–182.

[111] Haoye Tian, Kui Liu, Abdoul Kader Kaboré, Anil Koyuncu, Li Li, Jacques Klein, and Tegawendé F. Bissyandé. 2020. Evaluating representation learning of code changes for predicting patch correctness in program repair. In *Proceedings of the Automated Software Engineering*. ACM, Virtual Event Australia, 981–992.

[112] Haoye Tian, Xunzhu Tang, Andrew Habib, Shangwen Wang, Kui Liu, Xin Xia, Jacques Klein, and Tegawendé F. Bissyandé. 2022. Is this change the answer to that problem?: Correlating descriptions of bug and code changes for evaluating patch correctness. In *Proceedings of the Automated Software Engineering*. ACM, Rochester MI USA, 1–13.

[113] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 1–29.

[114] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., 5998–6008.

[115] Shangwen Wang, Ming Wen, Bo Lin, Hongjun Wu, Yihao Qin, Deqing Zou, Xiaoguang Mao, and Hai Jin. 2020. Automated patch correctness assessment: How far are we?. In *Proceedings of the Automated Software Engineering*. ACM, Virtual Event, 968–980.

[116] Thomas Wang, Adam Roberts, Daniel Hesslow, Teven Le Scao, Hyung Won Chung, Iz Beltagy, Julien Launay, and Colin Raffel. 2022. What language model architecture and pretraining objective work best for zero-shot generalization?. In *Proceedings of the International Conference on Machine Learning*. 22964–22984.

[117] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[118] Zhenmei Shi, Junyi Wei, Zhuoyan Xu, and Yingyu Liang. 2024. Why larger language models do In-context learning differently? In *International Conference on Machine Learning*. PMLR, 44991–45013.

[119] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-aware patch generation for better automated program repair. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg Sweden, 1–11.

[120] Chu-Pan Wong, Priscila Santiesteban, Christian Kästner, and Claire Le Goues. 2021. VarFix: Balancing edit expressiveness and search effectiveness in automated program repair. In *Proceedings of the ESEC/FSE '21*. Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.), ACM, 354–366.

[121] Ga Wu, Masoud Hashemi, and Christopher Srinivasa. 2022. PUMA: Performance unchanged model augmentation for training data removal. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 8 (2022), 8675–8682.

[122] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering* . IEEE, Melbourne, Australia, 1482–1494.

[123] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In *Proceedings of the ESEC/FSE*. ACM, Singapore Singapore, 959–971.

[124] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 1–10.

[125] Tongtong Xu, Liushan Chen, Yu Pei, Tian Zhang, Minxue Pan, and Carlo A. Furia. 2022. Restore: Retrospective fault localization enhancing automated program repair. *IEEE Transactions on Software Engineering* 48, 1 (2022), 309–326.

[126] Xuezheng Xu, Yulei Sui, Hua Yan, and Jingling Xue. 2019. VFix: Value-flow-guided precise program repair for null pointer dereferences. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering* . IEEE, Montreal, QC, Canada, 512–523.

[127] Chen Yang. 2021. Accelerating redundancy-based program repair via code representation learning and adaptive patch filtering. In *Proceedings of the ESEC/FSE*. ACM, Athens Greece, 1672–1674.

[128] Deheng Yang, Kui Liu, Dongsun Kim, Anil Koyuncu, Kisub Kim, Haoye Tian, Yan Lei, Xiaoguang Mao, Jacques Klein, and Tegawendé F. Bissyandé. 2021. Where were the repair ingredients for Defects4j bugs? *Empir. Softw. Eng.* 26, 6 (2021), 122.

[129] Deheng Yang, Xiaoguang Mao, Liqian Chen, Xuezheng Xu, Yan Lei, David Lo, and Jiayu He. 2022. TransplantFix: Graph differencing-based code transplantation for automated program repair. In *Proceedings of the ASE 2022, Rochester, MI, USA, October 10-14, 2022*. ACM, 107:1–107:13.

[130] He Ye, Jian Gu, Matias Martinez, Thomas Durieux, and Martin Monperrus. 2022. Automated classification of overfitting patches with statically extracted code features. *IEEE Transactions on Software Engineering* 48, 8 (2022), 2920–2938.

[131] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. SelfAPR: Self-supervised program repair with test execution diagnostics. In *Proceedings of the Automated Software Engineering*. ACM, Oakland Center, MI, US, 1–13.

[132] He Ye, Matias Martinez, and Martin Monperrus. 2021. Automated patch assessment for program repair at scale. *Empir. Softw. Eng.* 26, 2 (2021), 20.

[133] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural program repair with execution-based backpropagation. In *Proceedings of the International Conference on Software Engineering*. ACM, Pittsburgh, PA, US, 1506–1518.

[134] Zhongxing Yu, Matias Martinez, Benjamin Danglot, Thomas Durieux, and Martin Monperrus. 2019. Alleviating patch overfitting with automatic test generation: A study of feasibility and effectiveness for the nopol repair system. *Empir. Softw. Eng.* 24, 1 (2019), 33–67.

[135] Yuan Yuan and Wolfgang Banzhaf. 2020. ARJA: Automated repair of java programs via multi-objective genetic programming. *Transactions on Software Engineering* 46, 10 (2020), 1040–1067.

[136] Haibo Zhang, Toru Nakamura, Takamasa Isohara, and Kouichi Sakurai. 2023. A review on machine unlearning. *SN Computer Science* 4, 4 (2023), 337.

[137] Jiyang Zhang, Sheena Panthaplackel, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2022. CoditT5: Pretraining for source code and natural language editing. In *Proceedings of the Automated Software Engineering*. ACM, Rochester MI USA, 1–12.

[138] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. arXiv:2303.18223 [cs]

[139] Wenkang Zhong, Hongliang Ge, Hongfei Ai, Chuanyi Li, Kui Liu, Jidong Ge, and Bin Luo. 2022. StandUp4NPR: Standardizing setup for empirically comparing neural program repair systems. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Rochester MI USA, 1–13.

[140] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the ESEC/FSE*. ACM, Athens Greece, 341–353.

[141] Qihao Zhu, Zeyu Sun, Wenjie Zhang, Yingfei Xiong, and Lu Zhang. 2023. Tare: Type-aware neural program repair. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering*. IEEE, Melbourne, Australia, 1443–1455.

Received 20 January 2024; revised 29 August 2024; accepted 3 November 2024